University of Colombo

Department of Physics

Computational Physics Laboratory
CPL 110 Monte Carlo Methods I

Name- H.M.T.H. Lenadora

Reg num- 2022s19441

Index num- s16820

Contents

1.Introduction 3
2.1 Question 1 (Random number distribution histogram) 4
2.1.1 The objective 4
2.1.3. How it works 5
2.1.5 Discussion on results 6
2.2 Question 2 (Temperature converter) 6
2.2.1 Objective 6
2.2.3 How it works 8
2.2.5 Discussion on results 12
2.3.2 The code developed 13
2.3.3. How it works 14
2.3.4. Results 15
2.3.5. Discussion on results 15
2.4 Question 4 (Area calculation) 16
2.4.2 The code developed 17
2.4.3. How it works 18
2.4.4. Results 19
2.4.5. Discussion on results 19
2.5 Question 5 (Area calculation for graph) 20
2.5.1 Objective 20
2.5.2 The code developed 20
2.5.3. How it works 21
2.5.5 Discussion on results 21
2.6 Question 6 (current cross a resistor) 22
2.6.1 Objective 22
2.6.3. How it works 23
2.6.4 Results 23
2.6.5 Discussion on results 24
2.8.3. How it works 29
2.8.5 Discussion on results 30
2.9.1 Objective 31
2.10.1 Objective 34

Figure 1-code for random number histogram 4
Figure 2- results for random number histogram 5
Figure 3- code for Question 2 each part 8
Figure 4- results for Question 2 for each 12
Figure 5- code for calculate value varies 14
Figure 6- results for calculated value varies 15
Figure 7-code for area calculation 18
Figure 8-results for area calculation 19
Figure 9-area calculation code 20
Figure 10-results for area calculation 21
Figure 11- code for current across a resistor 23
Figure 12- results histogram for current across a resistor 24
Figure 13- code for voltage divider 25
Figure 14- results for voltage divider 26
Figure 15- code for Integration 29
Figure 16- results for integration 30
Figure 17-code for Customer Shopping Behavior in a Mall 32
Figure 18-Customer Shopping Behavior in a Mall 33
Figure 19- code for snake and ladder board game simulation 35
Figure 20- results for snake and ladder game 36

1.Introduction

This week's lab focuses on learning and applying the Monte Carlo method to solve a variety of problems in science, engineering, and real-life scenarios. The Monte Carlo method is a statistical technique that uses repeated random sampling to estimate results, especially when exact solutions are difficult or impossible to obtain. In this lab, tasks include generating and analyzing random numbers, understanding Z-scores for fair comparisons, estimating mathematical constants like π, calculating areas between curves, and solving integration problems. You will also explore practical applications such as resistor and voltage divider tolerances, as well as simulations like customer behavior, and games like Snake and Ladder using Markov Chain Monte Carlo. These exercises aim to develop your skills in programming, data visualization, and interpreting simulation results to make informed decisions.

2.Methods, Results and Discussion

2.1 Question 1 (Random number distribution histogram)

2.1.1 The objective

The objective of this task is to generate a set of random numbers that are uniformly distributed within the range of -1 to +1 and visualize their distribution using a histogram. A uniform distribution means that all values in the range have an equal probability of occurring. By plotting the histogram, we can visually inspect whether the random numbers are spread evenly across the range. This step helps in understanding how random number generation works in Python and how to verify the nature of the generated data. It also builds a foundation for using randomness in simulations and statistical methods.

2.1.2 The code developed

```
# Q1
```

```
import numpy as np
import matplotlib.pyplot as plt

# Generate 1000 random numbers uniformly distributed between -1 and 1
x = np.random.uniform(-1, 1, 1000)

# Plot histogram
plt.hist(x, bins=30, edgecolor='black')
plt.title('Histogram of Uniformly Distributed Random Numbers (-1 to 1)')
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.grid(True)
plt.show()
```

Figure 1-code for random number histogram

2.1.3. How it works

This code generates 1000 random numbers uniformly distributed between -1 and 1 using the np.random.uniform function. The x array stores these values, ensuring that each number within the range has an equal chance of appearing. The plt.hist function is then used to create a histogram, dividing the range into 30 bins to show how often numbers fall into each interval. The histogram provides a visual representation of the data's distribution, helping us check whether it appears uniform. Labels, a title, and a grid are added for clarity, and plt.show() displays the final plot on the screen.

2.1.4. Results

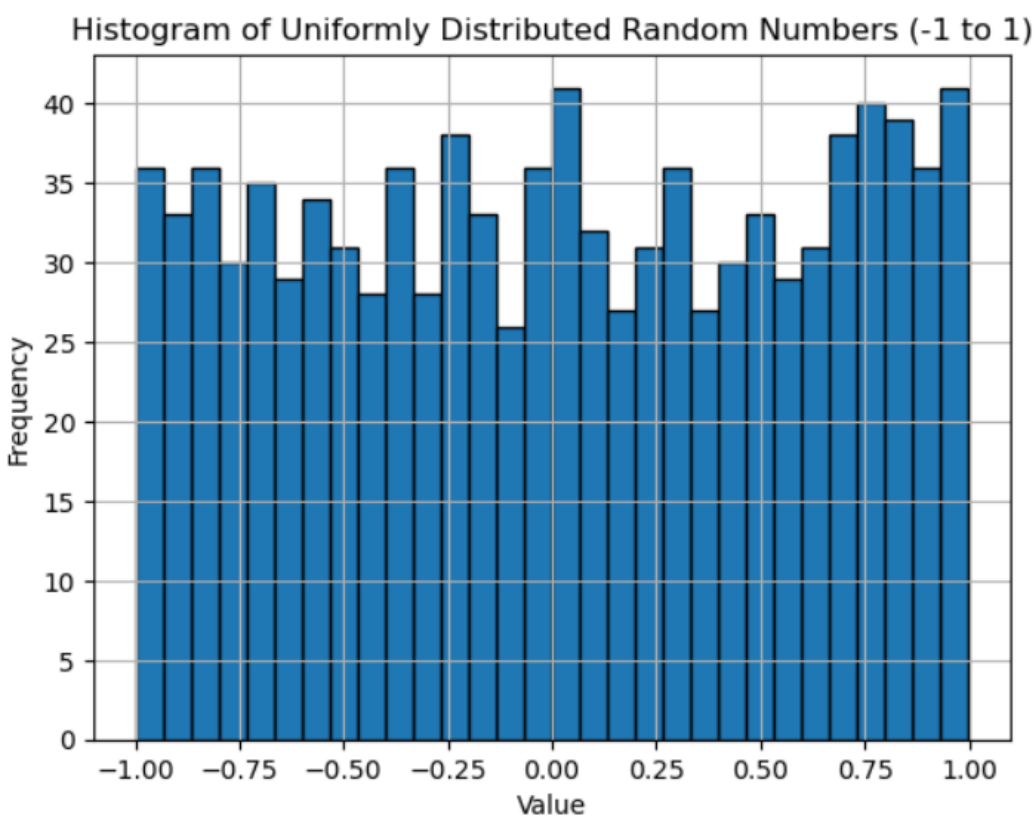


Figure 2- results for random number histogram

2.1.5 Discussion on results

The histogram produced from the generated random numbers clearly demonstrates the characteristics of a uniform distribution in the range of -1 to 1. The bars are relatively similar in height, meaning that each subrange within the given limits contains approximately the same number of values. This is expected because the np.random.uniform function gives each number in the range an equal probability of being selected. Small fluctuations in bar heights occur naturally due to randomness, especially with only 1000 samples, but these variations do not indicate any systematic bias. If the sample size were increased, the histogram would appear smoother and more evenly balanced, approaching the ideal shape of a perfectly flat distribution. This confirms that the method used generates values that align well with theoretical expectations of uniform distribution. The result also demonstrates the importance of visualization when verifying data randomness, as it allows us to detect patterns, anomalies, or deviations from the expected distribution.

2.2 Question 2 (Temperature converter)

2.2.1 Objective

The objective of this exercise is to use Monte Carlo simulation to generate a realistic set of exam marks for 1000 students and to study how standardization with Z-scores affects selection decisions. You will create marks from a normal distribution with a given mean and standard deviation, then apply truncation to keep marks within the feasible range of 0–100. By computing the sample mean and standard deviation and converting raw marks to Z-scores, you will inspect the transformed distribution and confirm that Z-scores have mean zero and unit variance. The task also asks you to compare raw marks and Z-scores when selecting the top students, and to investigate a two-paper scenario where exam difficulty differs; this highlights issues of fairness and comparability. Through plotting histograms and scatter graphs, you will learn how sampling variability, truncation, and standardization influence conclusions and practical decision making.

2.2.2 The code developed

```
# Q2  1000 numbers from a normal distribution with a mean of 50 and a standard deviation 30
```

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import truncnorm

# Set random seed for reproducibility
np.random.seed(42)

# Normal distribution (mean = 50, std = 30)
raw_marks = np.random.normal(50, 30, 1000)

# Plot histogram
plt.hist(raw_marks, bins=30, edgecolor='black')
plt.title('Histogram of Physics Marks (Unclipped)')
plt.xlabel('Marks')
plt.ylabel('Number of Students')
plt.grid(True)
plt.show()
```

[ ]:
```
# Modifed limit the distribution to the range of 0 to 100 using a set of truncated Gaussian
```

[3]:
```
# Parameters
mean = 50
std = 30
lower, upper = 0, 100

# Get truncated normal distribution
a, b = (lower - mean) / std, (upper - mean) / std
truncated_marks = truncnorm.rvs(a, b, loc=mean, scale=std, size=1000)

# Plot histogram
plt.hist(truncated_marks, bins=30, edgecolor='black')
plt.title('Histogram of Truncated Physics Marks (0 to 100)')
plt.xlabel('Marks')
plt.ylabel('Number of Students')
plt.grid(True)
plt.show()
```

```
[4]: # calculated mean and the standard devition
```

```
[5]: # Print mean and standard deviation of raw marks
print("Raw Marks - Mean:", np.mean(raw_marks))
print("Raw Marks - Std Dev:", np.std(raw_marks))

# Print mean and standard deviation of truncated marks
print("Truncated Marks - Mean:", np.mean(truncated_marks))
print("Truncated Marks - Std Dev:", np.std(truncated_marks))
```

```
]: #  Calculate Z-scores
```

```
]: mean_trunc = np.mean(truncated_marks)
std_trunc = np.std(truncated_marks)

z_scores = (truncated_marks - mean_trunc) / std_trunc
#print(z_scores)
# Plot Z-score distribution
plt.hist(z_scores, bins=30, edgecolor='black')
plt.title('Z-score Distribution of Physics Marks')
plt.xlabel('Z-score')
plt.ylabel('Number of Students')
plt.grid(True)
plt.show()
```

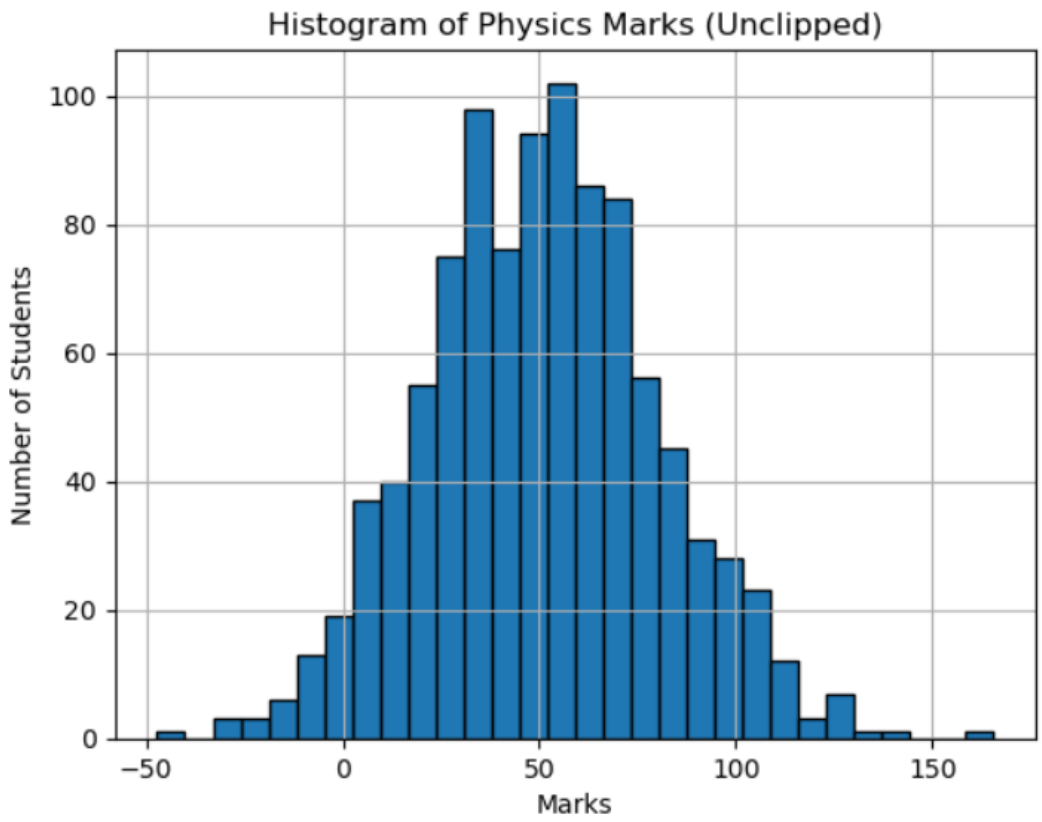
Figure 3- code for Question 2 each part

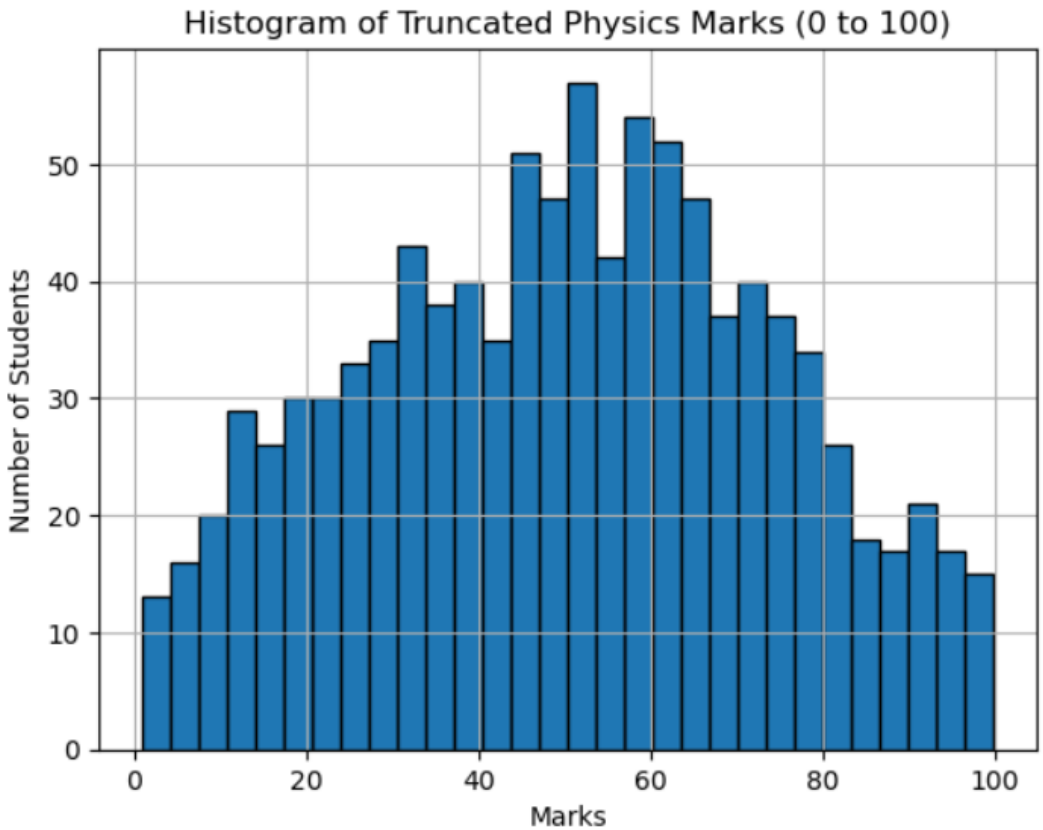
2.2.3 How it works

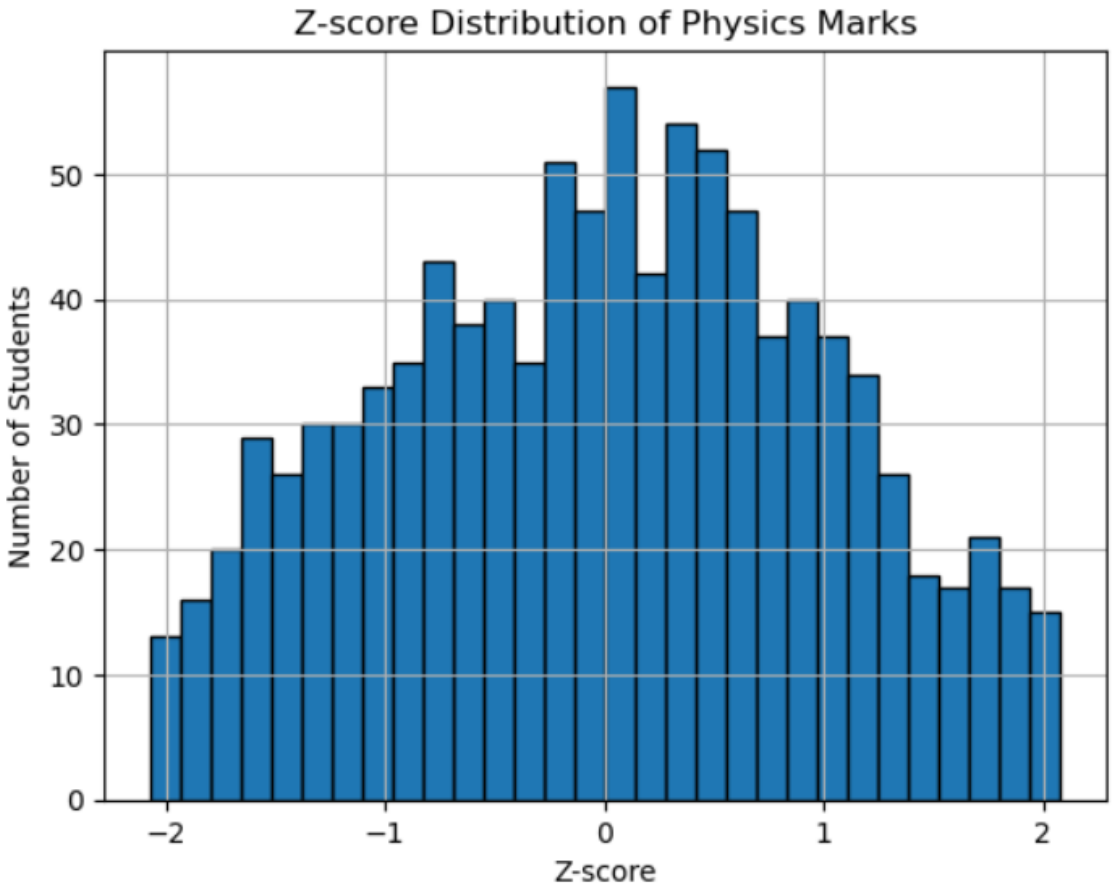
This code generates a simulated set of exam marks, adjusts them to a realistic range, and standardizes them using Z-scores for analysis. First, it creates 1000 random marks from a normal distribution with a mean of 50 and a standard deviation of 30, which can include unrealistic values below 0 and above 100. A histogram is plotted to visualize this raw distribution. To correct the data, a truncated normal distribution is generated so that all marks fall between 0 and 100 while keeping the general shape of the original distribution. Another histogram shows the truncated marks, and the mean and standard deviation are calculated for both the raw and truncated sets to observe changes caused by truncation. The code then calculates Z-scores by subtracting the truncated mean from each mark and dividing by the truncated standard deviation, producing a standardized dataset with mean 0 and standard deviation 1. Finally, a histogram of

Z-scores is plotted to illustrate the transformed distribution, making it easier to compare students fairly regardless of the original scale.

2.2.4 Results




Histogram of Truncated Physics Marks (0 to 100)
Number of Students
Marks
0
10
20
30
40
50
0
20
40
60
80
100


Z-score Distribution of Physics Marks
Number of Students
Z-score
0
10
20
30
40
50
−2
−1
0
1
2

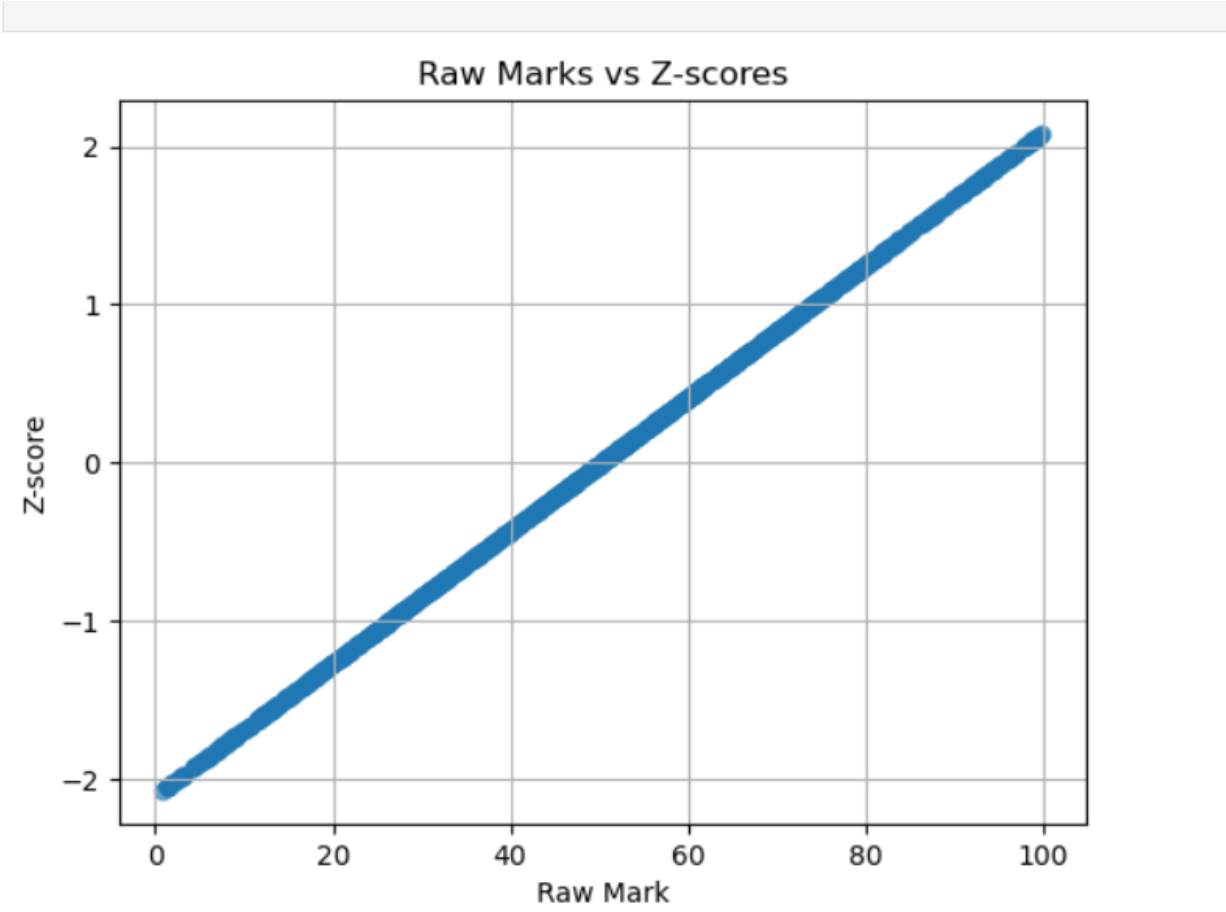


Figure 4- results for Question 2 for each

2.2.5 Discussion on results

The results show that the raw marks generated from the normal distribution follow the expected bell-shaped curve but include unrealistic values below 0 and above 100. This demonstrates that a normal distribution without constraints can produce outliers that are not meaningful in the context of exam scores. After applying the truncated normal distribution, all marks fall within the valid range, and the histogram still retains a similar shape to the original, although the mean and standard deviation shift slightly due to the removal of extreme values. The Z-score transformation produces a standardized distribution with a mean of zero and a standard deviation of one, making it independent of the original scale of marks. This confirms that Z-scores allow fair comparison between different datasets, which is particularly important when exam difficulty

varies. The clear correlation between raw marks and Z-scores indicates that while the ranking of students remains the same within the same dataset, Z-scores become essential when comparing results from different groups or papers.

2.3 Question 3 (calculated value varies)

2.3.1 Objective

The objective of this task is to estimate the value of Pi using the Monte Carlo method, which is a statistical technique based on random sampling. By generating random points within a square and counting how many fall inside the quarter circle inscribed within that square, we can approximate Pi from the ratio of points inside the circle to the total number of points. The goal is to observe how the accuracy of this estimation improves as the number of random points increases. This helps to understand the relationship between sample size and the precision of Monte Carlo simulations, illustrating the trade-off between computational effort and result accuracy.

2.3.2 The code developed

```
[1]: # Q3
```

```
[2]: import numpy as np
import matplotlib.pyplot as plt

# Number of random points (change this value to see accuracy)
N = 10000

# Generate N random points (x, y) in the square [0, 1] x [0, 1]
x = np.random.rand(N)
y = np.random.rand(N)

# Check if each point is inside the quarter circle (x^2 + y^2 <= 1)
inside = x**2 + y**2 <= 1

# Estimate pi using the ratio of points inside the circle
pi_estimate = 4 * np.sum(inside) / N

# Plotting
plt.figure(figsize=(6,6))
plt.scatter(x[inside], y[inside], color='blue', s=1, label='Inside Circle')
plt.scatter(x[~inside], y[~inside], color='red', s=1, label='Outside Circle')
plt.title(f'Estimating Pi using Monte Carlo Method\nEstimated Pi ≈ {pi_estimate:.5f}, N = {
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True)
plt.axis('square')
plt.show()
```

Figure 5- code for calculate value varies

2.3.3. How it works

First, the program generates a large number of random points within a square that ranges from 0 to 1 along both the x and y axes. Then, it checks which of these points lie inside a quarter circle of radius 1 by calculating if the sum of the squares of their coordinates is less than or equal to 1. The ratio of points inside the quarter circle to the total number of points approximates the area of the quarter circle compared to the square. Since the area of the quarter circle is π/4\pi/4, multiplying this ratio by 4 gives an estimate of Pi. The program also visually plots the points inside the circle in blue and those outside in red to help illustrate the method. This approach shows how increasing the number of points improves the accuracy of the Pi estimate.

2.3.4. Results

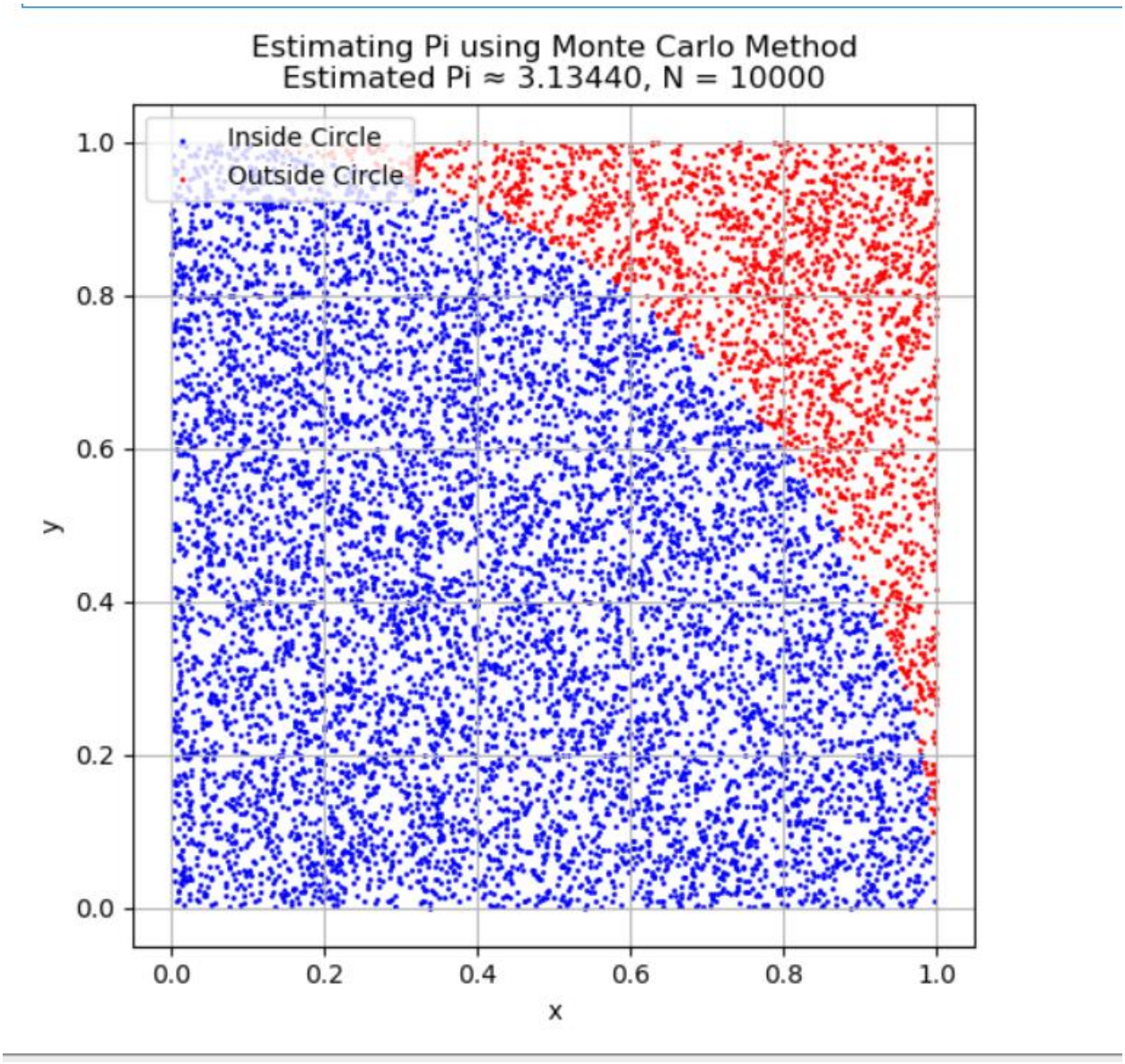


Figure 6- results for calculated value varies

2.3.5. Discussion on results

The Monte Carlo method uses random sampling to estimate the value of Pi by approximating the area of a quarter circle inside a unit square. As the number of random points increases, the estimate of Pi becomes more accurate because the statistical fluctuations decrease. With a small number of points, the estimated value may vary widely due to randomness, which can lead to a

poor approximation. However, as more points are used, the ratio of points inside the quarter circle approaches the true area ratio, and the estimate converges closer to the actual value of Pi (approximately 3.14159). This behavior shows the law of large numbers in action, where larger sample sizes yield better estimates. The scatter plot visually confirms this by showing a clearer division between points inside and outside the quarter circle when more points are generated. However, increasing the number of points also increases computation time, so there is a trade-off between accuracy and efficiency. Overall, the method demonstrates how Monte Carlo simulations can approximate complex mathematical constants using random sampling and statistical analysis.

2.4 Question 4 (Area calculation)

2.4.1 The objective

The objective of this task is to estimate the area of the region enclosed between two given curves using the Monte Carlo method. By randomly generating points within a defined rectangular box and determining how many points lie between the two curves, we can approximate the area based on the ratio of these points to the total number of points. This method provides a practical approach to calculate areas where finding the exact integral might be complex. Additionally, the task involves visualizing the points to better understand the distribution and discussing ways to improve the accuracy of the area estimation, such as increasing the number of random points or refining the sampling method.

2.4.2 The code developed

[1]:
```
# Q4
```

[2]:
```
import random
import matplotlib.pyplot as plt

def upper_curve(x):
    return -2.1 * x**2 + 2 * x + 0.5

def lower_curve(x):
    return -0.1 * x + 0.5

def estimate_area_monte_carlo(total_points):

    count_inside = 0
    x_inside = []
    y_inside = []
    x_outside = []
    y_outside = []

    x_start, x_end = 0, 1
    y_start, y_end = 0, 1
    bounding_area = (x_end - x_start) * (y_end - y_start)

    for _ in range(total_points):
        rand_x = random.uniform(x_start, x_end)
        rand_y = random.uniform(y_start, y_end)

        if lower_curve(rand_x) <= rand_y <= upper_curve(rand_x):
            count_inside += 1
            x_inside.append(rand_x)
            y_inside.append(rand_y)
        else:
```

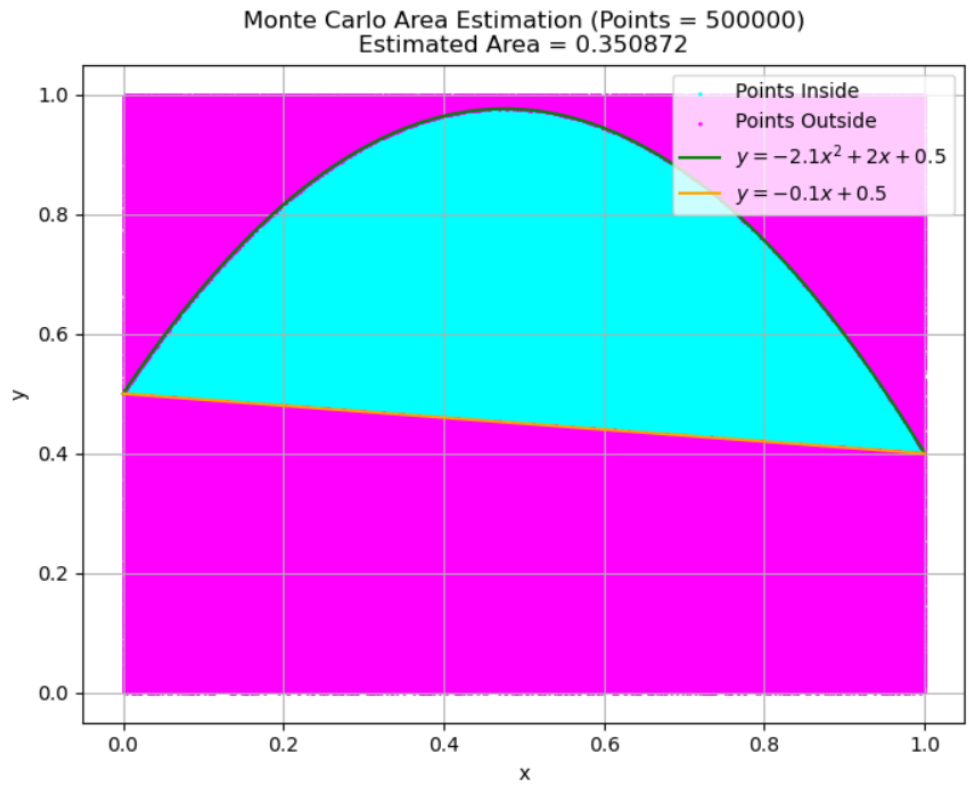
```python
            y_inside.append(rand_y)
        else:
            x_outside.append(rand_x)
            y_outside.append(rand_y)

    area_estimate = bounding_area * (count_inside / total_points)

    plt.figure(figsize=(8, 6))

    plt.scatter(x_inside, y_inside, color='cyan', s=1, label='Points Inside')
    plt.scatter(x_outside, y_outside, color='magenta', s=1, label='Points Outside')

    x_vals = [i / 100 for i in range(101)]
    plt.plot(x_vals, [upper_curve(x) for x in x_vals], color='green', label='$y = -2.1x^2 + 2x + 0.5$')
    plt.plot(x_vals, [lower_curve(x) for x in x_vals], color='orange', label='$y = -0.1x + 0.5$')

    plt.title(f'Monte Carlo Area Estimation (Points = {total_points})\nEstimated Area = {area_estimate:.6f}')
    plt.xlabel('x')
    plt.ylabel('y')
    plt.legend()
    plt.grid(True)
    plt.show()

    return area_estimate

if __name__ == '__main__':
    total_points = 500000
    area = estimate_area_monte_carlo(total_points)

    print(f"The estimated area of the region is: {area:.6f}")
```

Figure 7-code for area calculation

2.4.3. How it works

First, it defines two functions for the upper and lower curves based on the given equations. Next, it sets a bounding box from 0 to 1 on both the x and y axes, which contains the entire region between the curves. The program then generates a large number of random points inside this box, each with random x and y values. For each point, it checks whether the y-value lies between the values of the lower and upper curves at that x-position. Points that satisfy this condition are counted as inside the region. By finding the ratio of points inside the region to the total points generated and multiplying this ratio by the area of the bounding box (which is 1), the code estimates the area between the two curves. Finally, the points are plotted with different colors to visually show which are inside and outside the region, along with the curves themselves.

2.4.4. Results




```
The estimated area of the region is: 0.350872
```

Figure 8-results for area calculation

2.4.5. Discussion on results

The Monte Carlo method provides a simple and effective way to estimate the area of complex regions by using random sampling. In this case, random points were generated inside a known bounding box, and the fraction of points falling between the two curves was used to approximate the area. As the number of points increases, the estimate becomes more accurate because the random sampling better represents the true distribution within the region. The plot visually confirms this by showing points colored differently inside and outside the region, clearly highlighting the target area. However, since the method relies on randomness, smaller sample sizes can lead to noisy or less accurate estimates. Increasing the number of points improves precision but also requires more computation time. To further improve accuracy, techniques like stratified sampling or importance sampling could be used. Overall, the Monte Carlo method is a practical tool for area estimation when analytical solutions are difficult or impossible.

2.5 Question 5 (Area calculation for graph)

2.5.1 Objective

The objective of this task is to estimate the area of the region defined by the inequality $(x^2 + y^2 - 2x)^2 \leq 4(x^2 + 4y^2)$ using the Monte Carlo simulation method. By generating random points within the square region defined by −10<x<10 and −10<y<10, the simulation will determine the proportion of points that satisfy the given constraint. This proportion, multiplied by the total area of the bounding box, provides an estimate of the area of the region. Additionally, the task includes visualizing the region by plotting the points that satisfy the constraint, allowing for a clear graphical representation of the shape defined by the inequality.

2.5.2 The code developed

[2]:
```
import numpy as np
import matplotlib.pyplot as plt

# Number of random points
N = 100000

# Generate random (x, y) points in the square [-10, 10] × [-10, 10]
x = np.random.uniform(-10, 10, N)
y = np.random.uniform(-10, 10, N)

# Apply the inequality condition
left_side = (x**2 + y**2 - 2*x)**2
right_side = 4 * (x**2 + 4 * y**2)
inside = left_side <= right_side

# Estimate area
square_area = (20 * 20)  # total area of [-10,10] × [-10,10]
points_inside = np.sum(inside)
area_estimate = square_area * (points_inside / N)

# Plotting
plt.figure(figsize=(7,7))
plt.scatter(x[inside], y[inside], color='green', s=0.5, label='Inside Region')
plt.scatter(x[~inside], y[~inside], color='red', s=0.5, label='Outside Region')
plt.title(f'Estimated Area ≈ {area_estimate:.2f} using Monte Carlo\nPoints: {N}')
plt.xlabel('x')
plt.ylabel('y')
plt.axis('square')
plt.grid(True)
plt.legend()
plt.show()
```

Figure 9-area calculation code

2.5.3. How it works

First, it generates a large number of random points within the square region defined by -10 to 10 on both the x and y axes. For each point, it calculates the left and right sides of the inequality $(x^2 + y^2 - 2x)^2 \leq 4(x^2 + 4y^2)$ to check if the point lies inside the region. Points that satisfy the inequality are considered inside the region. The code counts how many points fall inside compared to the total number of points generated. Since the total area of the square is known (400), multiplying the ratio of points inside by this area gives an estimate of the region's area. Finally, the points inside and outside the region are plotted in different colors to visually show the shape, and the estimated area is displayed.

2.5.4 Results

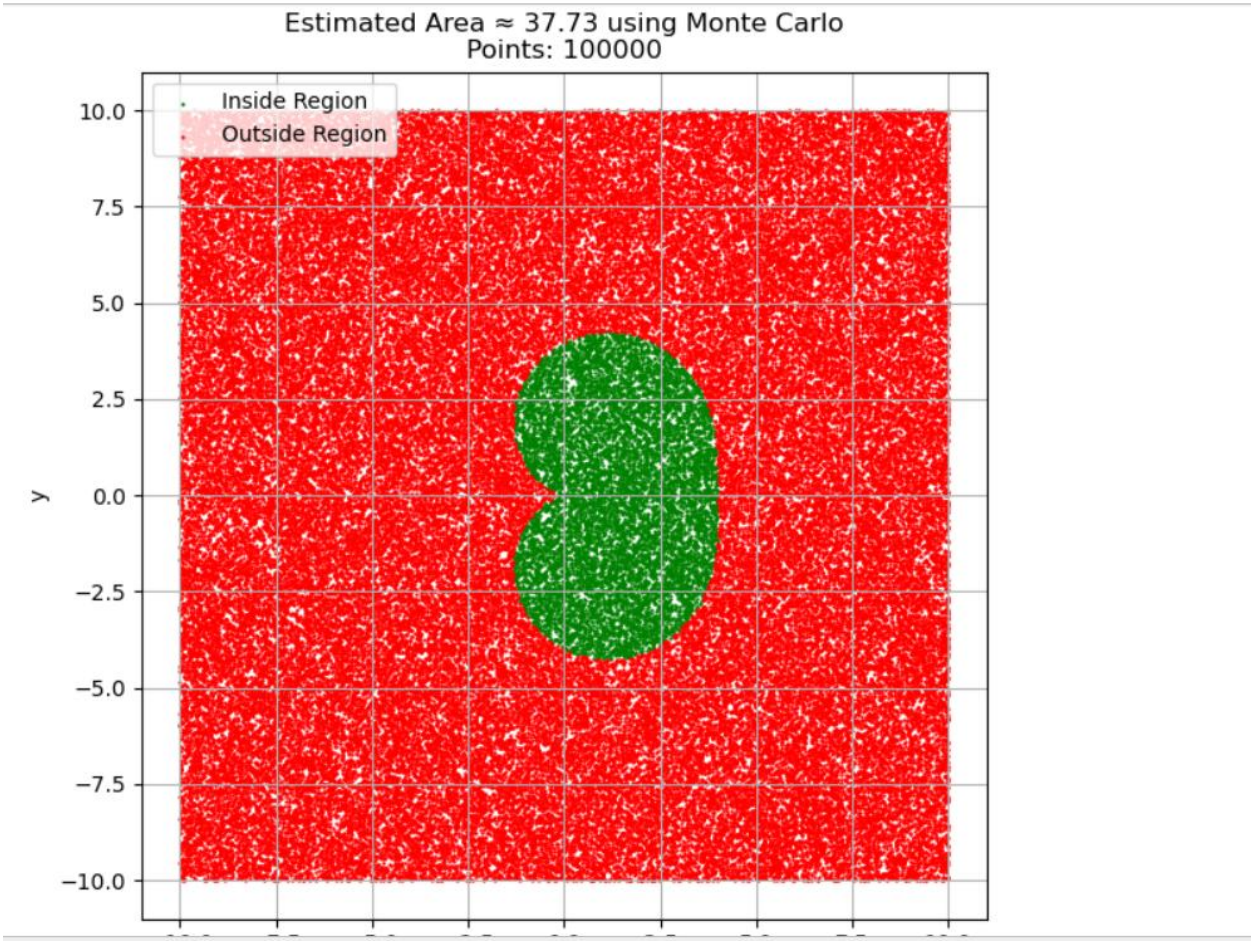


Figure 10-results for area calculation

2.5.5 Discussion on results

The Monte Carlo method provides an intuitive way to estimate the area of complex regions by random sampling. In this case, random points were generated inside a large square bounding

box, and the fraction of points satisfying the inequality defined the region of interest. As the number of points increased, the estimate of the area became more stable and closer to the true value. The plot visually confirmed the shape of the region by showing points inside in green and outside in red, illustrating the boundary clearly. However, because the method relies on randomness, smaller sample sizes can cause fluctuations in the estimate. Increasing the number of points improves accuracy but also increases computation time. To reduce variance without increasing points drastically, methods like stratified sampling or importance sampling could be considered. Overall, this approach effectively approximates the area of a region that would be difficult to compute analytically.

2.6 Question 6 (current cross a resistor)

2.6.1 Objective

The objective of this experiment is to simulate the statistical distribution of electrical components using the Monte Carlo method. Specifically, the task is to generate 1,000 resistance values for resistors with a nominal resistance of 470 Ω and a 5% tolerance. These values will be used to create a histogram, which visually represents the distribution of the resistance values. Finally, the corresponding current values will be calculated for each resistor, assuming a constant voltage, to demonstrate how component tolerance affects circuit performance.

2.6.2 The code developed

```
[2]: import numpy as np
     import matplotlib.pyplot as plt

     # Constants
     nominal_resistance = 470  # ohms
     tolerance = 0.05  # ±5%
     voltage = 12  # volts
     num_resistors = 1000

     # 1. Generate resistor values with 5% tolerance (uniform distribution)
     min_resistance = nominal_resistance * (1 - tolerance)
     max_resistance = nominal_resistance * (1 + tolerance)
     resistors = np.random.uniform(min_resistance, max_resistance, num_resistors)

     # 2. Calculate current for each resistor using Ohm's law: I = V / R
     currents = voltage / resistors

     # 3. Plot histogram of resistance values
     plt.figure(figsize=(10, 4))
     plt.subplot(1, 2, 1)
     plt.hist(resistors, bins=30, edgecolor='black', color='orange')
     plt.title('Resistance Distribution (470Ω ±5%)')
     plt.xlabel('Resistance (Ω)')
     plt.ylabel('Number of Resistors')
     plt.grid(True)

     # 4. Plot histogram of current values
     plt.subplot(1, 2, 2)
     plt.hist(currents, bins=30, edgecolor='black', color='skyblue')
     plt.title('Current Through Resistors (12V)')
     plt.xlabel('Current (A)')
     plt.ylabel('Number of Resistors')
     plt.grid(True)
```

Figure 11- code for current across a resistor

2.6.3. How it works

First, it sets the nominal resistance value of 470 ohms, the tolerance of ±5%, the voltage of 12 volts, and the number of resistors to simulate (1000). Next, it generates 1000 resistance values uniformly spread between 446.5 ohms and 493.5 ohms, representing the tolerance range around the nominal value. For each resistor, the current is calculated using Ohm's law by dividing the fixed voltage (12V) by the resistance value. Then, the code plots two histograms side by side: one showing how the resistance values are distributed within the tolerance range, and the other showing the resulting distribution of current values through those resistors. Finally, the plots are displayed with clear titles, labels, and grid lines to help visualize the spread in resistance and how it affects current.

2.6.4 Results

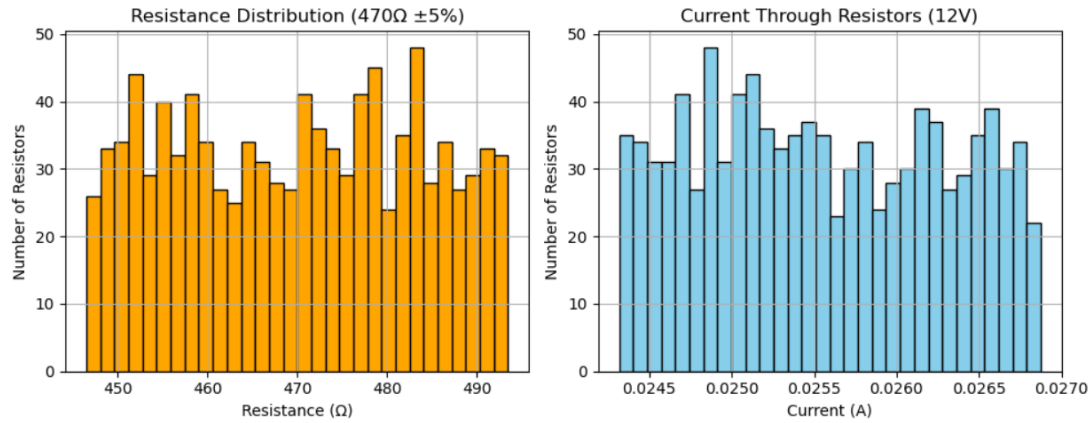


Figure 12- results histogram for current across a resistor

2.6.5 Discussion on results

The histogram of resistance values shows a uniform distribution between 446.5 Ω and 493.5 Ω, which matches the specified ±5% tolerance around the nominal 470 Ω value. This means the resistors in the sample vary evenly within the allowed range. The current histogram, on the other hand, shows a distribution that is not uniform but slightly skewed because current is inversely proportional to resistance. Resistors with lower resistance allow higher current, so the current distribution is stretched toward higher values corresponding to the lower end of the resistance range. Conversely, higher resistance values produce lower currents. This relationship is reflected clearly in the histograms. Overall, these results demonstrate how tolerance in component values affects circuit behavior, and why understanding these variations is important for designing reliable electronic circuits. Increasing the number of resistors would give smoother histograms and a better approximation of real-world variations.

2.7 Question 7 (Voltage divider circuit)

2.6.1 Objective

The core objective of this exercise is to model and analyze the statistical behavior of a voltage divider circuit using a Monte Carlo simulation. By generating thousands of random resistance values for resistors R_1 and R_2within their specified tolerance ranges, the simulation calculates the output voltage (Vout) for each combination. The primary goal is to then visualize the distribution of these calculated Vout values in a histogram, providing insight into the circuit's expected performance and variability under real-world conditions

2.7.2 The code developed

```
[2]: import numpy as np
     import matplotlib.pyplot as plt

     input_voltage = 10
     resistor1_nominal = 10000
     resistor2_nominal = 2000

     tolerance_r1 = 0.05
     tolerance_r2 = 0.02

     simulations = 100000

     r1_min = resistor1_nominal * (1 - tolerance_r1)
     r1_max = resistor1_nominal * (1 + tolerance_r1)
     r2_min = resistor2_nominal * (1 - tolerance_r2)
     r2_max = resistor2_nominal * (1 + tolerance_r2)

     r1_samples = np.random.uniform(r1_min, r1_max, simulations)
     r2_samples = np.random.uniform(r2_min, r2_max, simulations)

     v_out_samples = input_voltage * (r2_samples / (r1_samples + r2_samples))

     plt.figure(figsize=(10, 6))
     plt.hist(v_out_samples, bins=50, edgecolor='navy', alpha=0.8, color='lightgreen')
     plt.title(f'Monte Carlo Voltage Divider Output (Samples = {simulations})', fontsize=14)
     plt.xlabel('Output Voltage (V)')
     plt.ylabel('Count')
     plt.grid(True, linestyle='-.', alpha=0.7)
     plt.axvline(np.mean(v_out_samples), color='darkred', linestyle='dotted', linewidth=2,
                 label=f'Mean Output Voltage: {np.mean(v_out_samples):.4f} V')
     plt.legend()
     plt.tight_layout()
     plt.show()
```

Figure 13- code for voltage divider

2.7.3. How it works

First, it sets the input voltage, the nominal resistor values, their tolerances, and the number of simulations to run. Next, it calculates the minimum and maximum resistance values for each resistor based on their tolerances. Then, it generates many random resistor values uniformly distributed within these tolerance ranges. For each pair of resistor values, the code computes the output voltage of the voltage divider using the formula $V_{out} = V_{in} \times \frac{R_2}{R_1+R_{2-}}$ After that, it plots a histogram showing the distribution of output voltages resulting from these variations. Finally, the code calculates and marks the mean output voltage on the histogram to give a clear idea of the average expected output. This process helps visualize how resistor tolerances affect the output voltage in a real circuit.

2.7.4 Results

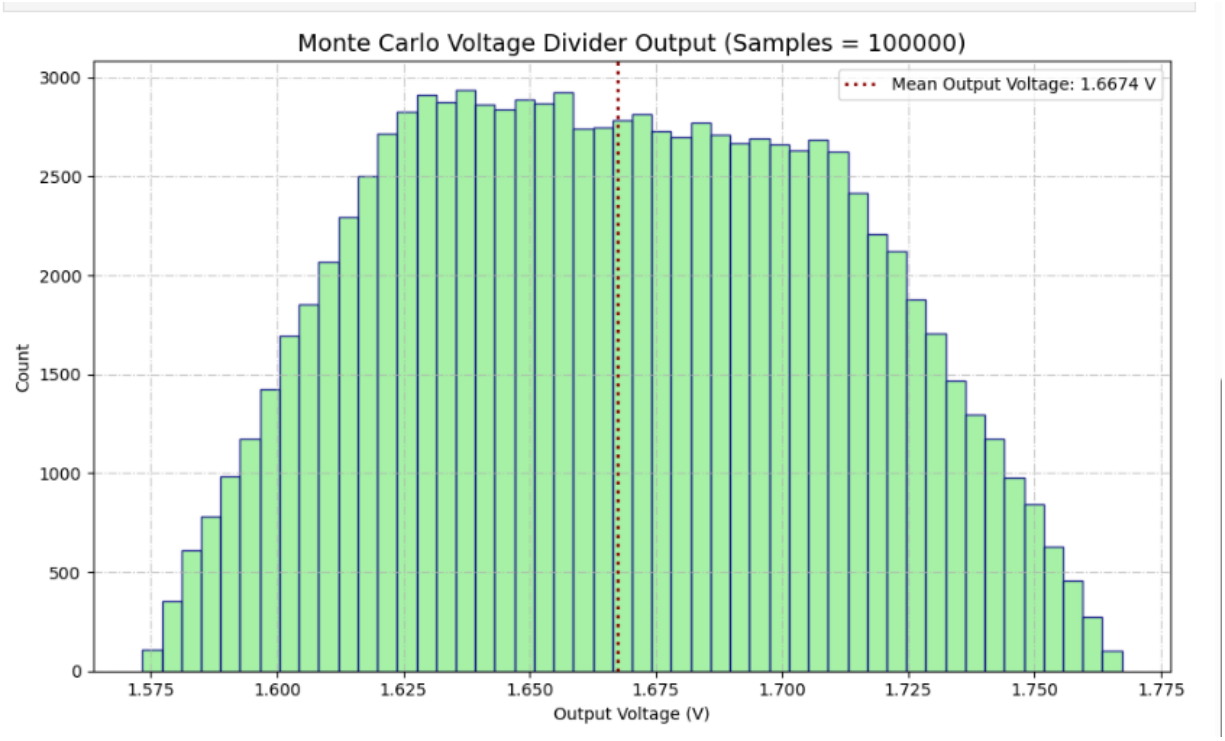


Figure 14- results for voltage divider

2.7.5 Discussion on results

The histogram shows how variations in resistor values due to their tolerances affect the output voltage. Because R1 and R2 are randomly varied within their tolerance ranges, the output voltage does not have a single fixed value but instead follows a distribution. The spread of the histogram reflects the combined effect of the resistors' tolerances on the output voltage. The mean output voltage, marked by the vertical line, indicates the average expected output under these variations. This result highlights that even small tolerance values can cause noticeable changes in circuit behavior. Understanding this variability is important for designing circuits that require precise voltage levels. Increasing the number of simulations improves the reliability of the estimated distribution and mean, while tighter resistor tolerances would reduce the spread of output voltages.

2.8 Question 8 (Integration)

2.8.1 Objective

The objective of this exercise is to implement and compare two different Monte Carlo methods for numerical integration. The primary goal is to calculate the area under the curve of the function sin(x) over the interval from 0 to π. The experiment involves two distinct approaches: a 1D integration method that uses random sampling to find the average function value, and a 2D method that estimates the area by counting random points that fall under the curve within a bounding box. The exercise concludes by comparing the results from both simulations against the known analytical solution to assess the accuracy and efficiency of each technique.

2.8.2 The code developed

[1]:
```
# Q8
```

[2]:
```
import numpy as np
import matplotlib.pyplot as plt

def f(x):
    return np.sin(x)

a = 0
b = np.pi

num_points = 500000

x_random_1d = np.random.uniform(a, b, num_points)
y_mean = np.mean(f(x_random_1d))
integral_1d = (b - a) * y_mean

y_min = 0
y_max = 1
area_of_box = (b - a) * (y_max - y_min)

x_random_2d = np.random.uniform(a, b, num_points)
y_random_2d = np.random.uniform(y_min, y_max, num_points)

points_under_curve = np.sum(y_random_2d <= f(x_random_2d))
integral_2d = area_of_box * (points_under_curve / num_points)

actual_value = 2.0

points_inside = y_random_2d <= f(x_random_2d)
x_inside = x_random_2d[points_inside]
y_inside = y_random_2d[points_inside]
```

```
actual_value = 2.0

points_inside = y_random_2d <= f(x_random_2d)
x_inside = x_random_2d[points_inside]
y_inside = y_random_2d[points_inside]
x_outside = x_random_2d[~points_inside]
y_outside = y_random_2d[~points_inside]

x_plot = np.linspace(a, b, 1000)
y_plot = f(x_plot)

plt.figure(figsize=(10, 6))
plt.plot(x_plot, y_plot, color='blue', label='y = sin(x)')
plt.scatter(x_inside, y_inside, color='green', s=1, label='Points inside')
plt.scatter(x_outside, y_outside, color='red', s=1, label='Points outside')
plt.title('2D Monte Carlo Integration')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
plt.xlim(a, b)
plt.ylim(y_min, y_max)
plt.show()

print(f"Number of random points: {num_points}")
print(f"Actual Integral of sin(x) from 0 to \u03C0: {actual_value}")
print(f"1D Monte Carlo Integration Result: {integral_1d:.6f}")
print(f"Error (1D): {abs(integral_1d - actual_value):.6f}")
print(f"2D Monte Carlo Integration Result: {integral_2d:.6f}")
print(f"Error (2D): {abs(integral_2d - actual_value):.6f}")
```

Figure 15- code for Integration

2.8.3. How it works

First, it defines the function sin(x) over the interval from 0 to π\pi. For the 1D Monte Carlo integration, it generates a large number of random xxx points uniformly distributed between 0 and π\piπ, computes the sine values at those points, averages them, and multiplies by the interval length to estimate the integral. For the 2D Monte Carlo method, it considers a rectangle bounding the curve with x from 0 to π\piπ and y from 0 to 1 (the maximum of sin(x)). It then generates random points inside this rectangle and counts how many points fall under the curve y≤sin(x). The ratio of points under the curve times the area of the rectangle estimates the integral. The code also plots the curve and colors the points inside the region under the curve in green, and points outside in red, showing visually how the random points cover the area. Finally,

it prints both the 1D and 2D Monte Carlo estimates, compares them to the actual integral value (2), and calculates the errors. This demonstrates two Monte Carlo approaches for integral estimation and highlights how random sampling approximates area under a curve.

2.8.4 Results

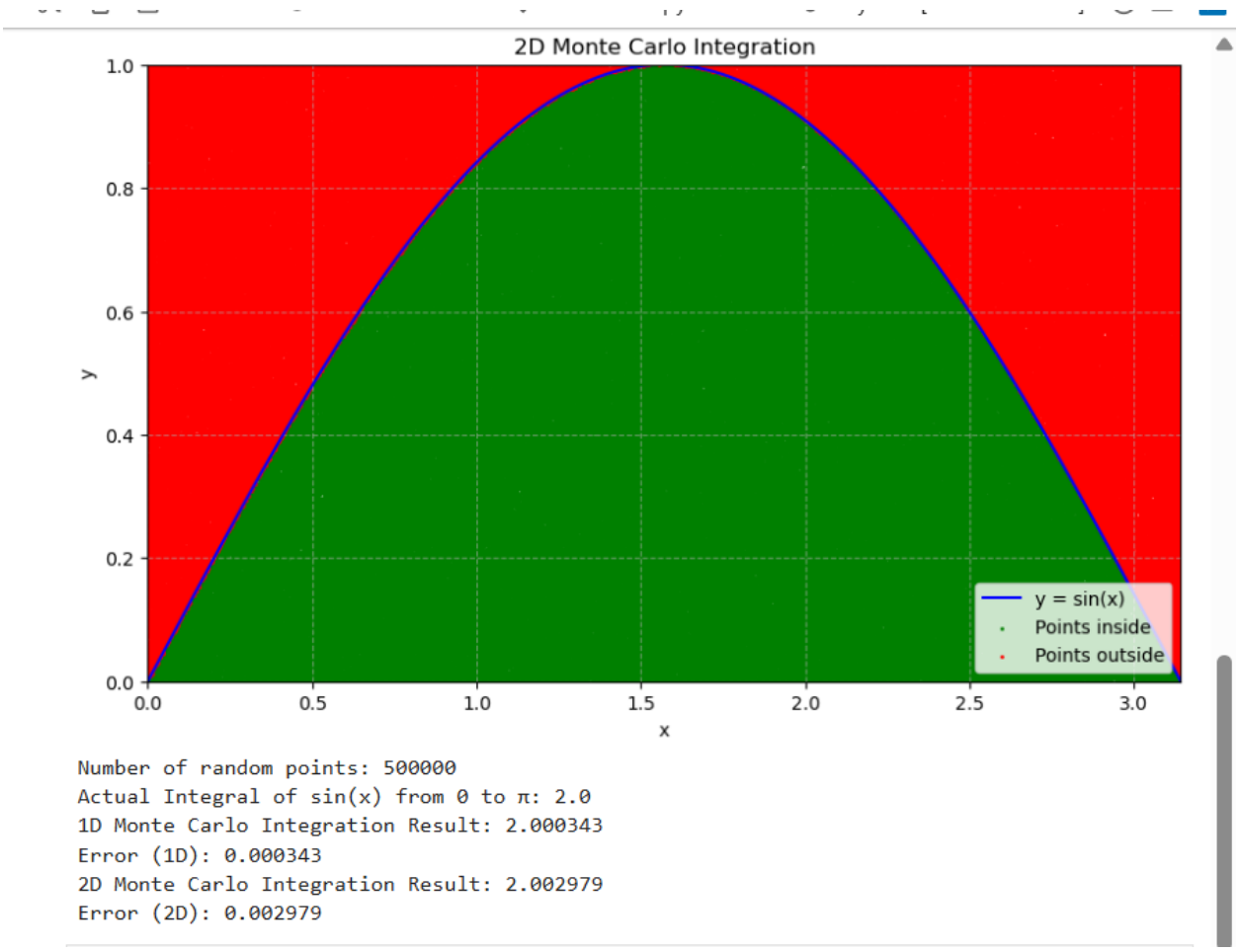


Figure 16- results for integration

2.8.5 Discussion on results

Both the 1D and 2D Monte Carlo methods provide approximate estimates of the integral of sin(x) from 0 to π\piπ. The actual value of this integral is exactly 2. The 1D method estimates the integral by averaging function values at randomly chosen points, while the 2D method estimates the integral by calculating the fraction of random points that lie under the curve inside a known bounding rectangle. Both methods produce results close to the true value, and the small

differences are due to random sampling variability. As the number of random points increases, these estimates tend to become more accurate and the errors reduce. The 2D method's visual plot helps intuitively understand how points under the curve relate to the integral. Overall, Monte Carlo integration is a useful numerical technique, especially for problems where analytical integration is difficult or impossible, though it generally requires a large number of samples for high accuracy.

2.9 Question 9 (Customer Shopping Behavior in a Mall)

2.9.1 Objective

The core objective of this exercise is to model and simulate customer shopping behavior within a mall using a Markov chain and the Monte Carlo method. Starting from the "Clothes" state, the simulation tracks a customer's movement between four defined states: Clothes, Electronics, Food, and Exit. By applying a fixed set of transition probabilities, the code simulates a large number of customer "walks" over various step counts. The final goal is to estimate the probability of a customer exiting the mall after a certain number of steps, thereby demonstrating how stochastic processes can be used to analyze and predict real-world decision-making patterns.

2.9.2 The code developed

```
[1]: # Q9

[2]: import numpy as np
import matplotlib.pyplot as plt

# Transition matrix for Markov chain (4 states, last is absorbing)
transition_matrix = np.array([
    [0.00, 0.50, 0.40, 0.10],
    [0.40, 0.00, 0.40, 0.20],
    [0.35, 0.50, 0.00, 0.15],
    [0.00, 0.00, 0.00, 1.00],
])

def simulate_exit_prob(num_steps, simulations=100_000, random_seed=None):
    if random_seed is not None:
        np.random.seed(random_seed)

    current_states = np.zeros(simulations, dtype=np.int8)  # start all at state 0

    for _ in range(num_steps):
        for state in (0, 1, 2):
            mask = (current_states == state)
            count = mask.sum()
            if count > 0:
                current_states[mask] = np.random.choice(4, size=count, p=transition_matrix[state])

    exit_prob = (current_states == 3).mean()
    return exit_prob

if __name__ == "__main__":
    steps_list = [5, 10, 20, 30]
    num_simulations = 200_000
```

```
        for state in (0, 1, 2):
            mask = (current_states == state)
            count = mask.sum()
            if count > 0:
                current_states[mask] = np.random.choice(4, size=count, p=transition_matrix[state])

    exit_prob = (current_states == 3).mean()
    return exit_prob

if __name__ == "__main__":
    steps_list = [5, 10, 20, 30]
    num_simulations = 200_000
    seed = 50

    exit_probabilities = []

    print(f"Simulations per step count = {num_simulations}, seed = {seed}\n")
    for steps in steps_list:
        prob = simulate_exit_prob(steps, simulations=num_simulations, random_seed=seed)
        exit_probabilities.append(prob)
        print(f"After {steps:2d} steps: estimated probability of exit = {prob:.5f}")

    # Plotting the results
    plt.figure(figsize=(8,5))
    plt.plot(steps_list, exit_probabilities, marker='o', linestyle='-', color='darkorange')
    plt.title('Estimated Exit Probability vs Number of Steps')
    plt.xlabel('Number of Steps')
    plt.ylabel('Probability of Exit (State 3)')
    plt.grid(True, linestyle='--', alpha=0.6)
    plt.ylim(0, 1)
    plt.show()
```

Figure 17-code for Customer Shopping Behavior in a Mall

2.9.3. How it works

First, the code defines a transition matrix that specifies the probabilities of moving from one state to another in a Markov chain with four states, where the last state is absorbing (once reached, the process stays there). It starts all simulated chains in state 0. Then, for each step up to the specified number, it updates the state of every chain currently in states 0, 1, or 2 by randomly

selecting the next state based on the corresponding row of the transition matrix. After all steps, it calculates the fraction of chains that have reached the absorbing exit state, giving an estimate of the exit probability by that step. The simulation repeats for different step counts to see how the exit probability evolves. Finally, it plots these probabilities against the number of steps to visualize the increasing likelihood of absorption over time. This Monte Carlo method effectively approximates the behavior of the Markov chain and its tendency to reach the absorbing state as time progresses.

2.9.4 Results

```
Simulations per step count = 200000, seed = 50

After  5 steps: estimated probability of exit = 0.80729
After 10 steps: estimated probability of exit = 0.96267
After 20 steps: estimated probability of exit = 0.99869
After 30 steps: estimated probability of exit = 0.99992
```

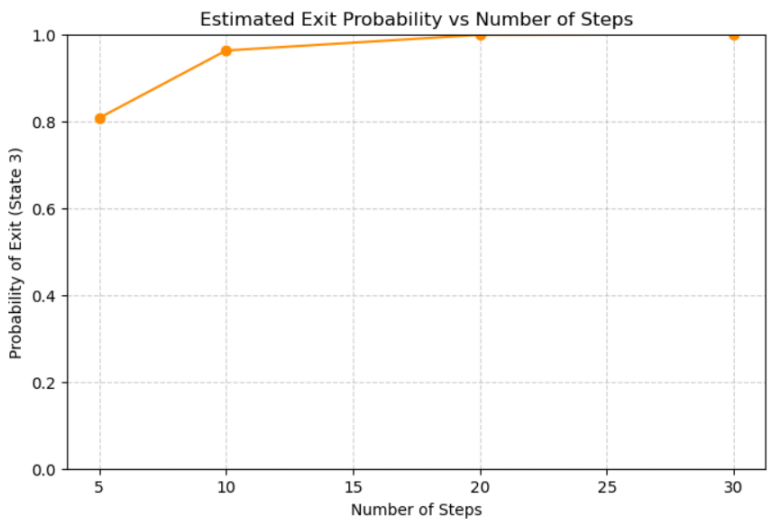


Figure 18-Customer Shopping Behavior in a Mall

2.9.5 Discussion on results

The results show how the probability of reaching the absorbing exit state increases as the number of steps grows. Initially, after a small number of steps, only a small fraction of chains have reached the exit, but as the simulation progresses, more chains transition into the absorbing state. This trend reflects the nature of Markov chains with absorbing states — given enough time, the probability of absorption approaches 1. The increasing exit probability with steps also illustrates how the chain "forgets" its starting state over time, converging toward the absorbing state. The simulation's large number of runs (simulations) helps ensure the estimated probabilities are

reliable and smooth. Overall, this demonstrates how Monte Carlo methods can effectively model and estimate complex stochastic processes like Markov chains, providing insights into their long-term behavior.

2.10 Question 10 (Snake and Ladder Board Game Simulation)

2.10.1 Objective

The core objective of this exercise is to simulate the dynamics of a simplified Snakes and Ladders game using a Monte Carlo approach. The simulation models the game as a Markov chain, beginning at the first square and progressing through a large number of games. Each game involves rolling a die, moving a player, and applying the rules for ladders, snakes, and overshooting the final square. The primary goal is to determine the average number of dice rolls required to complete the game. By analyzing these expected outcomes, the exercise provides insight into how chance and the board's design influence the game's overall flow and length.

2.10.2 The code developed

```python
[2]: import random
import numpy as np
import matplotlib.pyplot as plt

# Define ladders and snakes positions (start:end)
ladder_positions = {5: 14, 10: 28, 32: 45}
snake_positions = {15: 7, 37: 19, 49: 34}

def play_single_game():
    current_position = 1
    dice_rolls = 0

    while current_position < 50:
        dice_roll = random.randint(1, 6)
        dice_rolls += 1

        if current_position + dice_roll <= 50:
            current_position += dice_roll

            if current_position in ladder_positions:
                current_position = ladder_positions[current_position]
            elif current_position in snake_positions:
                current_position = snake_positions[current_position]
    return dice_rolls

def run_simulations(total_games=10000):
    rolls_per_game = [play_single_game() for _ in range(total_games)]

    average_rolls = np.mean(rolls_per_game)
    probability_finish_25_or_less = np.mean(np.array(rolls_per_game) <= 25)

    # Plot histogram
```

```python
                current_position = snake_positions[current_position]
    return dice_rolls

def run_simulations(total_games=10000):
    rolls_per_game = [play_single_game() for _ in range(total_games)]

    average_rolls = np.mean(rolls_per_game)
    probability_finish_25_or_less = np.mean(np.array(rolls_per_game) <= 25)

    # Plot histogram
    plt.figure(figsize=(10,5))
    plt.hist(rolls_per_game, bins=range(min(rolls_per_game), max(rolls_per_game)+1),
             color='mediumslateblue', edgecolor='black', alpha=0.75)
    plt.axvline(average_rolls, color='crimson', linestyle='dashed', linewidth=2,
                label=f'Average rolls: {average_rolls:.2f}')
    plt.title('Distribution of Rolls to Finish the Game')
    plt.xlabel('Number of Rolls')
    plt.ylabel('Frequency')
    plt.legend()
    plt.grid(True, linestyle='--', alpha=0.6)
    plt.show()

    return average_rolls, probability_finish_25_or_less

if __name__ == "__main__":
    total_games = 10000
    avg_rolls, prob_25_or_less = run_simulations(total_games)

    print(f"Simulated {total_games} games")
    print(f"Average number of rolls to finish the game: {avg_rolls:.2f}")
    print(f"Probability of finishing within 25 rolls: {prob_25_or_less:.4f}")
```

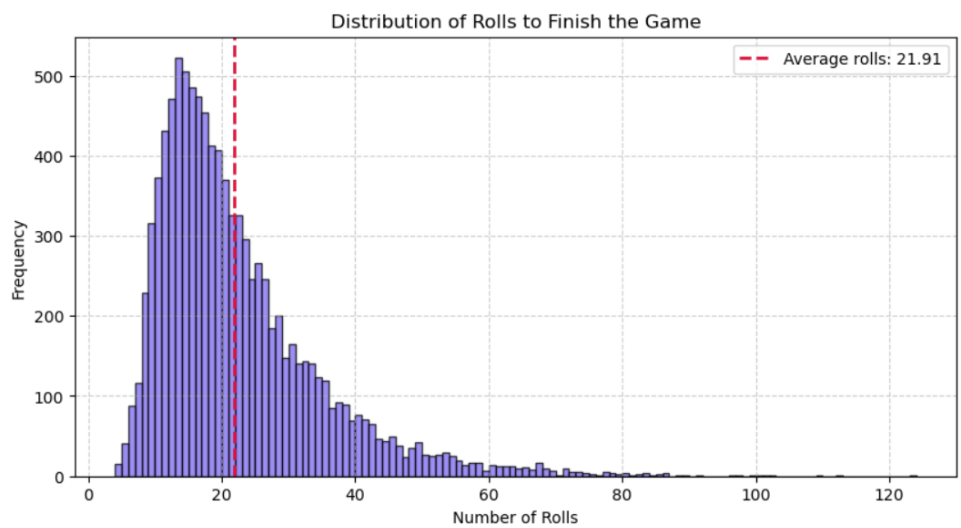
Figure 19- code for snake and ladder board game simulation

2.10.3. How it works

First, the code defines the locations of ladders and snakes on the board as mappings from start to

end positions. Then, it simulates a single game starting at position 1, rolling a dice (1 to 6) repeatedly. After each roll, it moves the player forward without exceeding position 50. If the new position matches a ladder's base, the player climbs up; if it matches a snake's head, the player slides down. This continues until the player reaches position 50. The number of dice rolls taken is recorded for that game. The simulation runs many such games (e.g., 10,000) to collect data on how many rolls are needed on average and how often the game finishes within 25 rolls. A histogram shows the distribution of rolls per game, with the average marked for reference. Finally, it prints the average rolls and the probability of finishing quickly. This Monte Carlo method captures the randomness and board effects to estimate game duration statistically.

2.10.4 Results




```
Simulated 10000 games
Average number of rolls to finish the game: 21.91
Probability of finishing within 25 rolls: 0.7169
```

Figure 20- results for snake and ladder game

2.10.5 Discussion on results

The results show the average number of dice rolls needed to complete the game, reflecting the combined effects of dice randomness, ladders, and snakes on game progress. The histogram illustrates the variability in game length, with some games finishing quickly due to favorable rolls and ladder climbs, while others take longer because of unlucky snake encounters or dice outcomes. The probability of finishing within 25 rolls provides insight into how often players

can expect a relatively fast game. This distribution highlights the inherent randomness of the game and how board features influence player movement. Overall, the simulation gives a realistic estimate of game duration and helps understand the balance between luck and progression in snakes and ladders.

3. Conclusion

In this report, we explored several applications of the Monte Carlo method, a powerful technique that uses random sampling to approximate solutions to complex problems. We began by estimating the value of Pi and calculating areas between curves, which demonstrated how Monte Carlo can approximate geometric quantities where exact calculation is difficult. Next, we applied the method to electrical circuits by simulating component tolerances and their effects on voltage and current, showing how randomness influences real-world systems.

We also studied Markov chains to estimate the probability of reaching certain states over time, highlighting the use of Monte Carlo in analyzing stochastic processes. Finally, we simulated the popular board game snakes and ladders to estimate the expected number of dice rolls to finish and the probability of finishing quickly. This illustrated how randomness and game elements like ladders and snakes affect gameplay duration.

Across all examples, increasing the number of random samples consistently improved the accuracy and reliability of results. The visualizations, including plots and histograms, helped to better understand the behavior of each system under uncertainty. Overall, Monte Carlo simulations proved to be a versatile and effective tool to model and analyze problems involving randomness, uncertainty, and probability in various scientific and engineering fields.